离线服务器的 Conda 环境离线分发

Victory-Hugo [1*]

[1] 重庆医科大学基础医学院，重庆市渝中区石油路，400016，中国

联系方式:
giantlinlinlin@gmail.com

[*] 第一作者

现地址: 重庆市渝中区石油路，400016，中国

摘要

以下步骤用于在可联网的 Linux 机器上，从现有 conda 环境生成适合人工传输到离线服务器的交付包。

介绍

快速命令汇总

先编辑 `conf/Config.json`，然后在联网机器上执行：

```
bash pipe/pipeline.sh export
```

然后：

1. 将生成的 `dist/XXXXX.tar.gz` 交付包传到离线服务器
2. 将 `conda 环境同步/conf`、`conda 环境同步/pipe`、`conda 环境同步/python`、`conda 环境同步/script`、`conda 环境同步/src` 目录也传到离线服务器，保持项目结构完整
3. 在离线服务器上执行：

```
bash pipe/pipeline.sh install \
```

项目结构

该项目按照生物信息学流水线规范组织：

- `conf/Config.json`：统一配置入口
- `pipe/pipeline.sh`：总控脚本
- `python/`：Python 双模式模块
- `script/`：shell 辅助脚本
- `src/`：预留给非 Python 模块

默认示例环境为 `ScikitAllele`。

目录说明

- result/exports/<run_id>/bundle/：离线交付目录
- dist/：最终 tar.gz 交付包
- logs/：运行日志
- tmp/：临时文件

联网机器用法

以下步骤用于在可联网的 Linux 机器上，从现有 conda 环境生成适合人工传输到离线服务器的交付包。

1. 联网机器前提

联网机器至少需要具备以下条件：

- 已安装 bash
- 已安装 python3
- 已安装 tar
- 已安装 sha256sum
- 已安装 conda
- 当前机器可以访问 conda 和 pip 所需的软件源

先确认基础命令可用：

```
conda --version
python3 --version
tar --version
sha256sum --version
```

2. 准备项目目录

本项目默认 project_dir 为：

```
/mnt/c/Users/Administrator/Desktop/conda 环境同步
```

在联网机器上进入项目目录：

```
cd /mnt/c/Users/Administrator/Desktop/conda 环境同步
```

确认目录中至少存在：

- conf/Config.json
- pipe/pipeline.sh
- python/
- script/

3. 确认目标 conda 环境存在

本项目默认示例环境是 `ScikitAllele`。先查看当前机器是否存在该环境：

```
conda env list
```

你应能看到类似结果：

```
ScikitAllele   /path/to/miniconda3/envs/ScikitAllele
```

如果没有这个环境，需要先在联网机器上把它准备好，再执行导出。

4. 检查导出配置

默认配置写在 `conf/Config.json`，关键字段如下：

```
"export": {
  "env_name": "ScikitAllele",
  "pip_download_mode": "prefer_binary",
  "healthcheck_imports": [
    "allel",
    "numpy",
    "pandas"
  ]
}
```

其中：

- `env_name`：要导出的 conda 环境名
- `pip_download_mode`：优先下载 wheel，失败后允许源码包
- `healthcheck_imports`：离线安装完成后要验证导入的 Python 包

如果你要导出别的环境，可以先修改 `conf/Config.json` 中的 `export.env_name`，再执行后续步骤。

5. 可选：先手动检查环境内容

为了确认当前环境确实是你想分发的版本，可以先检查：

```
conda list -n ScikitAllele | head -n 30
conda run -n ScikitAllele python -m pip freeze | head -n 30
```

如果你担心环境里混入了临时测试包、开发路径依赖或错误版本，建议先在联网机器上清理环境，再打包。

6. 执行导出

在项目根目录运行：

```
cd /mnt/c/Users/Administrator/Desktop/conda 环境同步
bash pipe/pipeline.sh export
```

该命令会自动完成以下动作：

- 导出 environment.yml
- 导出 conda list --explicit
- 导出 pip freeze
- 采集环境元数据
- 准备 conda 离线包缓存
- 准备 pip wheel 或源码包缓存
- 生成 manifest.json
- 生成 sha256.txt
- 生成最终 tar.gz 交付包

7. 查看导出日志

导出日志保存在：

```
logs/export-<timestamp>.log
```

例如：

```
ls logs
tail -n 50 logs/export-20260308T135737.log
```

如果导出失败，优先查看该日志。

8. 确认导出结果

导出完成后，主要产物包括：

- dist/*.tar.gz：适合人工传输的交付包
- result/exports/<run_id>/bundle/metadata/environment.yml
- result/exports/<run_id>/bundle/metadata/explicit.txt
- result/exports/<run_id>/bundle/metadata/pip_freeze.txt
- result/exports/<run_id>/bundle/metadata/pip_requirements_offline.txt
- result/exports/<run_id>/bundle/metadata/manifest.json
- result/exports/<run_id>/bundle/metadata/sha256.txt

以当前验证成功的一次导出为例，产物包括：

- dist/conda-env-offline-sync-ScikitAllele-20260308T135737.tar.gz
- result/exports/20260308T135737/bundle/

你可以直接查看：

```
ls dist
find result/exports -maxdepth 3 -type d | sort
```

9. 检查交付包内容

建议在传输前做一次快速人工检查：

```
tar -tzf dist/conda-env-offline-sync-ScikitAllele-20260308T135737.tar.gz | head -n
```

也可以直接查看元数据目录：

```
find result/exports/20260308T135737/bundle/metadata -maxdepth 1 -type f | sort
```

重点确认以下文件存在：

- `environment.yml`
- `explicit.txt`
- `pip_freeze.txt`
- `pip_requirements_offline.txt`
- `env_metadata.json`
- `manifest.json`
- `sha256.txt`
- `INSTALL.md`

10. 理解 `pip_requirements_offline.txt`

本项目会同时保留两份 pip 依赖文件：

- `pip_freeze.txt`
- `pip_requirements_offline.txt`

之所以要额外生成 `pip_requirements_offline.txt`，是因为某些 conda 环境中的 `pip freeze` 结果会包含本地构建路径，例如：

```
fasteners @ file:///home/conda/feedstock_root/...
```

这种写法无法直接在另一台离线服务器上重建。为此，导出流程会自动把这类条目改写成离线可重建的版本，例如：

```
fasteners==0.19
```

离线安装阶段会优先使用 `pip_requirements_offline.txt`。

11. 准备传输到离线服务器

推荐传输以下内容到离线服务器：

- 整个项目目录

- dist/ 下对应的 tar.gz 交付包

最稳妥的方式是直接复制整个项目目录，因为离线服务器执行安装时需要：

- conf/
- pipe/
- python/
- script/
- dist/

例如你可以把整个目录复制到移动硬盘或共享目录：

```
/mnt/c/Users/Administrator/Desktop/conda 环境同步
```

如果你只复制 tar.gz 而不复制项目本身，离线服务器上将缺少安装入口脚本，无法直接运行本项目。

12. 联网机器侧常见问题

EnvironmentLocationNotFound 或找不到 ScikitAllele

说明 conf/Config.json 中的 env_name 与实际环境名不一致，或环境尚未创建。

pip download 失败

通常是某些 pip 包缺少 wheel，或依赖来自本地路径。当前流程会自动尝试把本地路径型依赖规范化，并在 binary-only 失败后退回到允许源码包下载。

conda 包准备过慢

这是正常现象，特别是在首次导出、缓存未命中时。只要日志持续推进并最终生成 dist/*.tar.gz，即可认为导出成功。

sha256.txt 或 manifest.json 缺失

说明导出未完整结束，不能把该批次交付给离线服务器，应重新执行导出。

离线服务器安装步骤

以下步骤假定你已经在联网机器上生成好交付包，并通过 U 盘、共享盘或其他人工方式传到离线 Linux 服务器。

1. 离线服务器前提

离线服务器至少需要具备以下条件：

- 已安装 bash
- 已安装 python3
- 已安装 tar
- 已安装 sha256sum
- 已安装 conda

- 服务器平台应与导出机器兼容，至少要保持同类操作系统和 CPU 架构

先在离线服务器上确认 conda 可用：

```
conda --version
python3 --version
tar --version
sha256sum --version
```

如果 conda 命令不可用，需要先让服务器管理员安装 Miniconda 或 Anaconda，并确保当前 shell 能直接调用 conda。

2. 将项目与交付包放到离线服务器

建议把整个项目目录和导出的压缩包一起放到离线服务器，例如：

```
/data/project/conda 环境同步/
```

目录中至少应看到：

- conf/Config.json
- pipe/pipeline.sh
- python/
- script/
- dist/conda-env-offline-sync-ScikitAllele-<run_id>.tar.gz

例如：

```
cd /data/project/conda 环境同步
ls
```

如果你没有复制整个项目，而只复制了 bundle 或 tar.gz，则还不能直接运行安装流程。该项目的 pipe/、python/、script/、conf/ 必须与交付包一起存在。

3. 确认交付包名称

进入项目目录后，先确认你要安装的压缩包文件名：

```
cd /data/project/conda 环境同步
ls dist
```

例如你可能看到：

```
conda-env-offline-sync-ScikitAllele-20260308T135737.tar.gz
```

下文命令都以这个文件名为例。

4. 可选：先人工查看交付包内容

如果你希望在安装前做一次人工检查，可以先看压缩包内的文件结构：

```
tar -tzf dist/conda-env-offline-sync-ScikitAllele-20260308T135737.tar.gz | head -n
```

重点关注是否存在以下文件：

- bundle/metadata/environment.yml
- bundle/metadata/explicit.txt
- bundle/metadata/pip_requirements_offline.txt
- bundle/metadata/manifest.json
- bundle/metadata/sha256.txt

5. 配置安装目标位置

默认安装前缀写在 conf/Config.json：

```
"install": {
  "bundle_input": "",
  "target_prefix": "/home/luolintao/miniconda3/envs/ScikitAllele"
}
```

这表示如果你不传额外参数，环境会被安装到项目目录下的：

```
/home/luolintao/miniconda3/envs/ScikitAllele
```

建议在离线服务器上显式指定目标前缀，避免与已有目录冲突。例如：

```
/home/luolintao/miniconda3/envs/ScikitAllele
```

或

```
/data/conda_envs/ScikitAllele
```

注意：目标目录如果已经存在，安装会失败。这是有意设计，用来避免覆盖已有环境。

6. 执行离线安装

推荐用命令行显式传参，不依赖配置文件里的 bundle_input：

```
cd /data/project/conda 环境同步

bash pipe/pipeline.sh install \
  --bundle-path dist/conda-env-offline-sync-ScikitAllele-20260308T135737.tar.gz \
  --target-prefix /home/luolintao/miniconda3/envs/ScikitAllele
```

如果你希望安装到绝对路径，也可以这样写：

```
cd /data/project/conda 环境同步

bash pipe/pipeline.sh install \
```

```
  --bundle-path dist/conda-env-offline-sync-ScikitAllele-20260308T135737.tar.gz \
  --target-prefix /data/conda_envs/ScikitAllele
```

安装过程会自动完成以下工作：

- 解压交付包到 `tmp/`
- 校验 `sha256`
- 根据 `explicit.txt` 生成本地化的 `explicit_local.txt`
- 使用 `conda create --offline` 重建 conda 环境
- 使用本地 `wheelhouse` 执行 `pip install --no-index`
- 生成安装日志

7. 查看安装日志

安装日志保存在：

```
logs/install-<timestamp>.log
```

例如：

```
ls logs
tail -n 50 logs/install-20260308T135809.log
```

如果安装失败，优先查看这个日志。

8. 执行健康检查

安装完成后，立即运行健康检查：

```
cd /data/project/conda 环境同步

bash pipe/pipeline.sh health --target-prefix /home/luolintao/miniconda3/envs/Scikit
```

如果你安装到了绝对路径，则写绝对路径：

```
bash pipe/pipeline.sh health --target-prefix /data/conda_envs/ScikitAllele
```

默认会检查 `conf/Config.json` 中定义的导入项：

- `allel`
- `numpy`
- `pandas`

健康检查结果会写入：

```
logs/health-check-installed.json
```

你可以查看结果：

```bash
cat logs/health-check-installed.json
```

成功时应类似于：

```json
{
  "python_version": "3.10.20",
  "imports": {
    "allel": true,
    "numpy": true,
    "pandas": true
  },
  "errors": {}
}
```

9. 如果你想通过配置文件安装

虽然更推荐命令行显式传参，但也支持先改 `conf/Config.json` 再运行。

把：

```json
"install": {
  "bundle_input": "dist/conda-env-offline-sync-ScikitAllele-20260308T135737.tar.gz
  "target_prefix": "/home/luolintao/miniconda3/envs/ScikitAllele"
}
```

写入 `conf/Config.json` 后，直接执行：

```bash
bash pipe/pipeline.sh install
```

然后执行：

```bash
bash pipe/pipeline.sh health
```